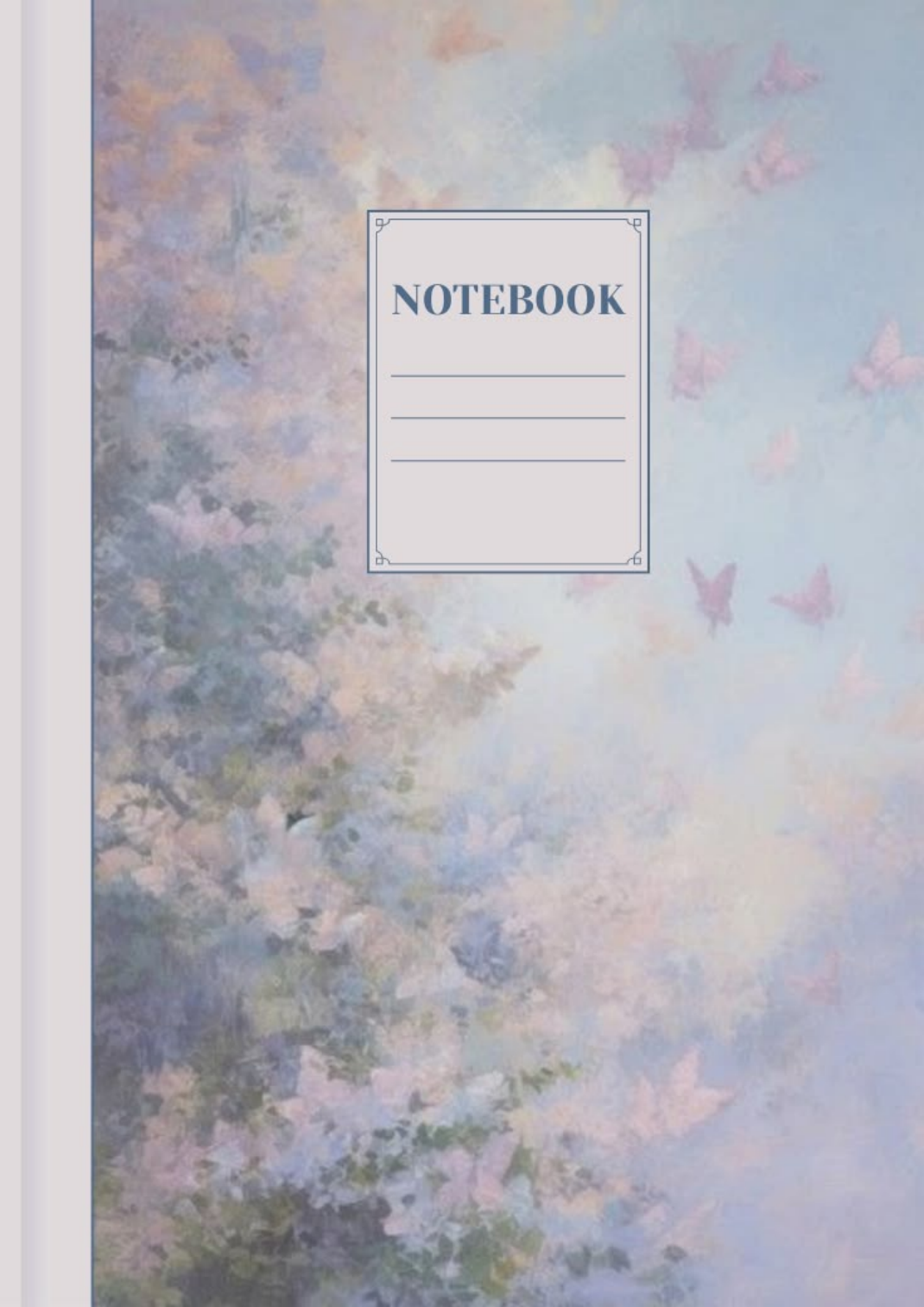
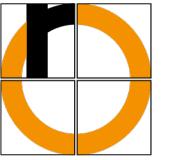




NOTEBOOK



Theoretical Computer Science

Introduction

Technische Hochschule Rosenheim
Sommer 2026
Prof. Dr. Jochen Schmidt

- Lecture: 2 SWS (= 2x 45min per week)
- Exercises: 2 SWS, split into groups
 - in English (mainly for AAI students) & German (mainly for INF students)
 - register via Learning Campus
- Workload: 150h (60h lecture/exercises, **90h additionally for studying on your own**)
- Slides, exercises etc.: Download via Learning Campus
- Exam
 - written exam at the end of the semester
 - 90 minutes
 - aids: non-programmable calculator (no other devices allowed)

Chapter 1: **Finite Automata** (*Endliche Automaten*)

- Definition and representation of automata
- Recognized language of automata

Chapter 2: **Pushdown Automata & Turing Machines** (*Kellerautomaten & Turing-Maschinen*)

- Pushdown automata
- Linear bounded automata
- Turing machines

Chapter 3: **Formal Languages** (*Formale Sprachen*)

- Definition of formal languages
- Chomsky hierarchy
- Pumping lemma

Chapter 4: **Word Problem & Parsing** (*Wortproblem & Parsing*)

- Analysis of words in formal language classes
- CYK-Parser
- Compiler

Chapter 5: **Computability Theory** (*Berechenbarkeit*)

- Decidability and Church-Turing thesis
- Halting problem
- LOOP/WHILE/GOTO computability
- primitive & μ -recursive recursive functions

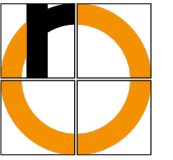
Chapter 6: **Complexity Theory** (*Komplexitätstheorie*)

- Time and space complexity
- Order of complexity, O-notation
- Optimization example: Divide-and-conquer
- Complexity classes P, NP
- NP-Completeness
- NP-hard Problems & other problem classes

Chapter 7: **Probabilistic Algorithms** (*Probabilistische Algorithmen*)

- Random numbers
- Monte-Carlo methods
- Probabilistic primality tests

- in German
 - H. Ernst, J. Schmidt, G. Beneken. **Grundkurs Informatik**. Springer Vieweg, 8. Aufl. 2023.
 - J. Schmidt. **Grundkurs Informatik – Das Übungsbuch: 163 Aufgaben mit Lösungen**. Springer Vieweg, 3. Auflage 2023.
 - L. Priese und K. Erk: **Theoretische Informatik. Eine umfassende Einführung**. Springer Vieweg, 4. Aufl., 2018.
 - D.W. Hoffmann. **Theoretische Informatik**. Hanser, 5. Aufl., 2022.
- in English
 - M. Sipser: **Introduction to the Theory of Computation**. Cengage Learning, Inc; 3. Edition 2021.
 - J.E. Hopcroft, R. Motwani und J.D. Ullmann: **Introduction to Automata Theory, Languages, and Computation**. Pearson Education Limited, 2013.
- Further Reading in English & German
 - B. Hollas: *Grundkurs Theoretische Informatik: Mit Aufgaben und Anwendungen*. Springer Vieweg, 2. Aufl., 2015.
 - G. Vossen und K.-U. Witt: *Grundkurs Theoretische Informatik: Eine anwendungsbezogene Einführung*. Springer Vieweg, 6. Aufl., 2016.
 - U. Schöning, U.: *Theoretische Informatik – kurz gefasst*. Spektrum Akad. Verlag, 5. Aufl., 2008.
 - A. Aho, M. Lam und R. Sethi. *Compilers*. Addison-Wesley Longman, 2013.
 - D.E. Knuth: *The Art of Computer Programming, Volumes 1-4A*. Addison-Wesley, 2011.
 - W.H. Press et al.: *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. Cambridge University Press, 3. Aufl., 2007.
 - R. Sedgewick: *Algorithms*. Addison-Wesley, 4. Aufl., 2011.
 - R. Sedgewick und P. Flajolet. *An Introduction to the Analysis of Algorithms*. Addison Wesley, 2. Aufl., 2013.



Theoretical Computer Science

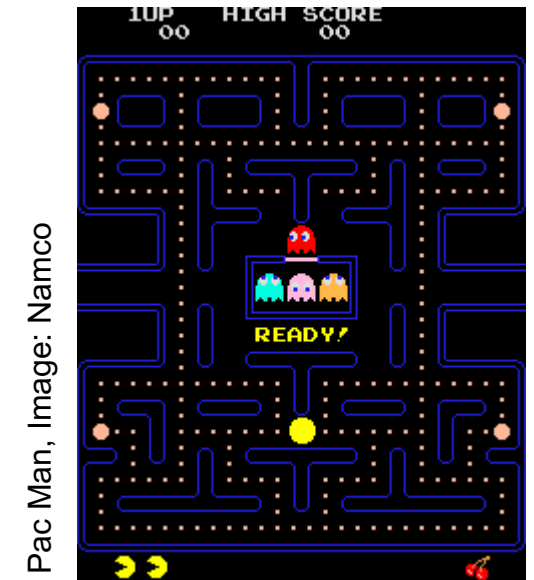
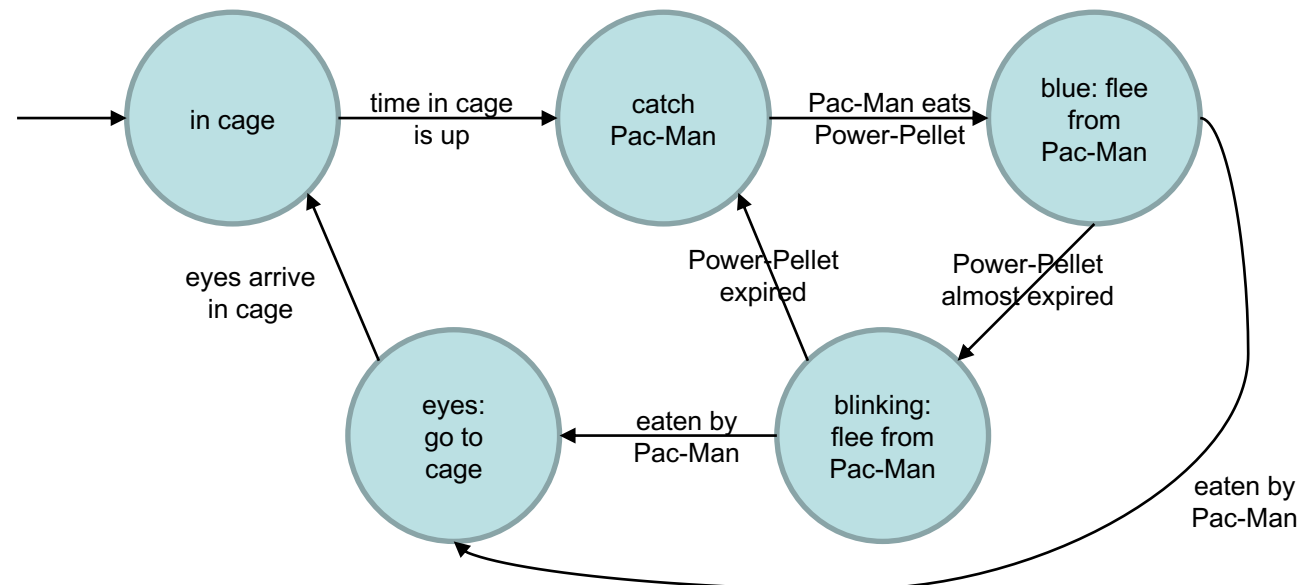
Finite Automata

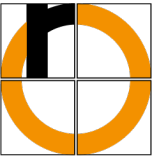
Technische Hochschule Rosenheim
Sommer 2026
Prof. Dr. Jochen Schmidt

- Definition and representation of finite automata
- Recognized language of accepting automata
- Deterministic & nondeterministic automata
- Minimal automata

- Automaton in layman's terms:
A machine that can control its behavior to a certain degree
 - e.g., a coffee dispenser (in German: Kaffee**automat**)
 - for applications in science and technology:
a more mathematical, precise definition/abstraction is required
- Applications in computer science and related fields
 - analysis and formal representation of complex dependencies and processes
 - used for modelling in software engineering
 - define states (*Zustände*) and transitions between states
 - close connection to computability theory and formal languages
 - and therefore a part of the fundamentals of programming languages and compilers
 - electrical engineering: design of integrated circuits
 - minimization of number of states and logical gates
 - optimization of connections

- Model states and transitions using a deterministic finite automaton
- very useful & clear representation
 - which state transitions are allowed and when?
 - many algorithms available for finite automata
- Example: Ghosts in Pac-Man



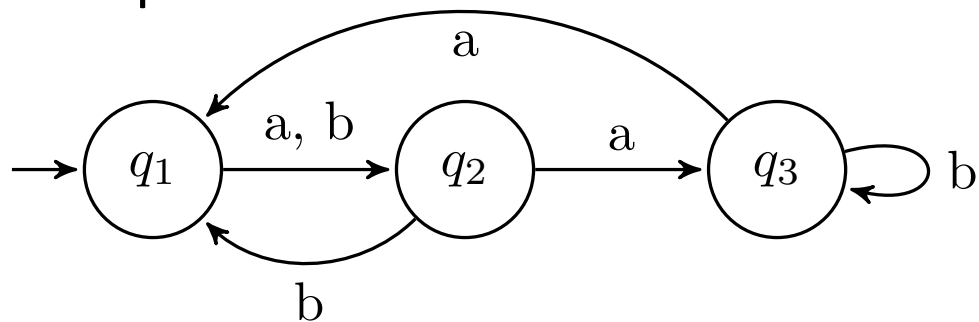


- **State diagrams** are directed graphs:

- vertices = states
- edges = transitions
- outdegree of each vertex = #symbols in input alphabet
- start state is marked by an ingoing arrow

Reminder:
Alphabet: set of single symbols

Example:



input sequence: string

⚠ have to indicate which is the start state (not always q_1)

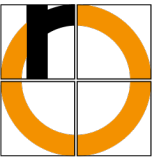
- **Transition tables**

- in a DFA, the number of possible transitions is limited to: nz (n : #states, z : #symbols)
- we can use a finite table to define the transition function

columns: states

σ_i	q_1	q_2	q_3
a	q_2	q_3	q_1
b	q_2	q_1	q_3

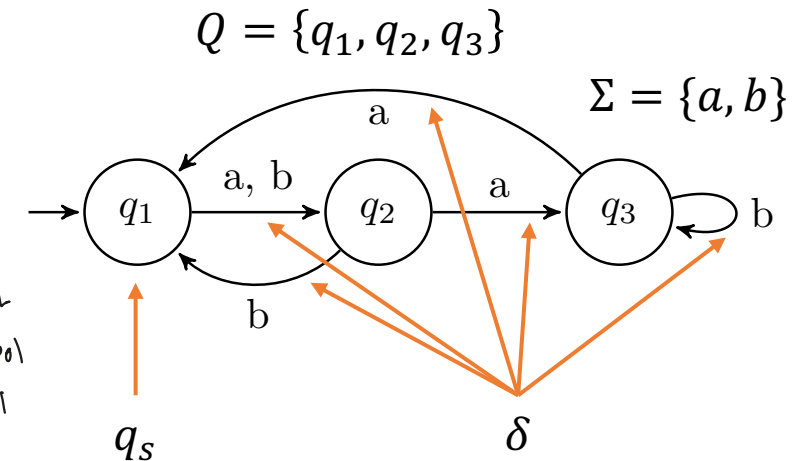
if you get a certain symbol in a certain state $\rightarrow$ what is the next state



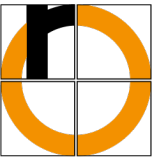
A deterministic automaton (*deterministischer Automat*) is defined by:

- A countable set (*abzählbare Menge*) of states (Zustände) $Q = \{q_1, q_2, \dots\}$
could be infinite but we restrict to finite
- An Alphabet (i.e., a countable, ordered set) of input symbols $\Sigma = \{\sigma_1, \sigma_2, \dots\}$
could be infinite but we restrict to finite
- A transition function (Zustandsübergangsfunktion), mapping state-input pairs to the next state $\delta: Q \times \Sigma \rightarrow Q$, where $Q \times \Sigma$ is the cartesian product of two sets, i.e., the set of all ordered pairs (q, σ) with $q \in Q$ and $\sigma \in \Sigma$
 - written as: $\delta(\overset{\text{state}}{q_i}, \overset{\text{input}}{\sigma_j}) = \overset{\text{output state}}{q_k}$ or $q_i \sigma_j \rightarrow q_k$
 - successor state is unique (therefore: deterministic)
 - but not necessarily invertible

*- where I am at the moment
- what's the next input symbol
- no information about where I came from*
- A start state $q_s \in Q$
- If Q and Σ are finite sets: deterministic finite automaton (DFA)
 - other terms: finite-state machine or just state machine



Exercise



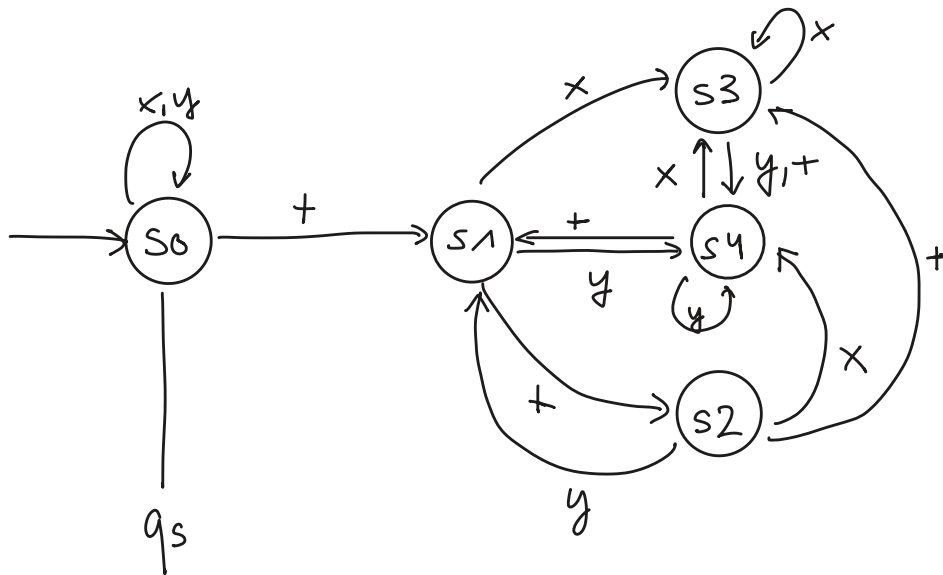
Draw the state diagram of an automaton having the following transition table:

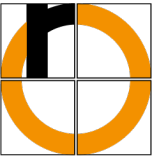
Alphabet = $\{x, y, +\}$

States = $\{s_0, s_1, s_2, s_3, s_4\}$

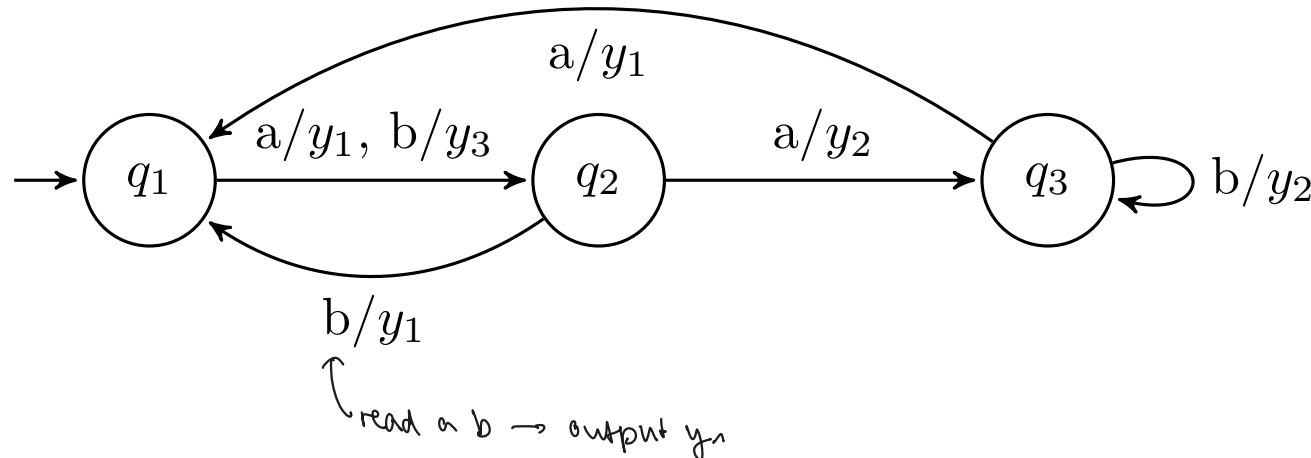
Start state = s_0

	s0	s1	s2	s3	s4
x	s0	s3	s4	s3	s3
y	s0	s4	s1	s4	s4
+	s1	s2	s3	s4	s1

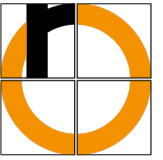




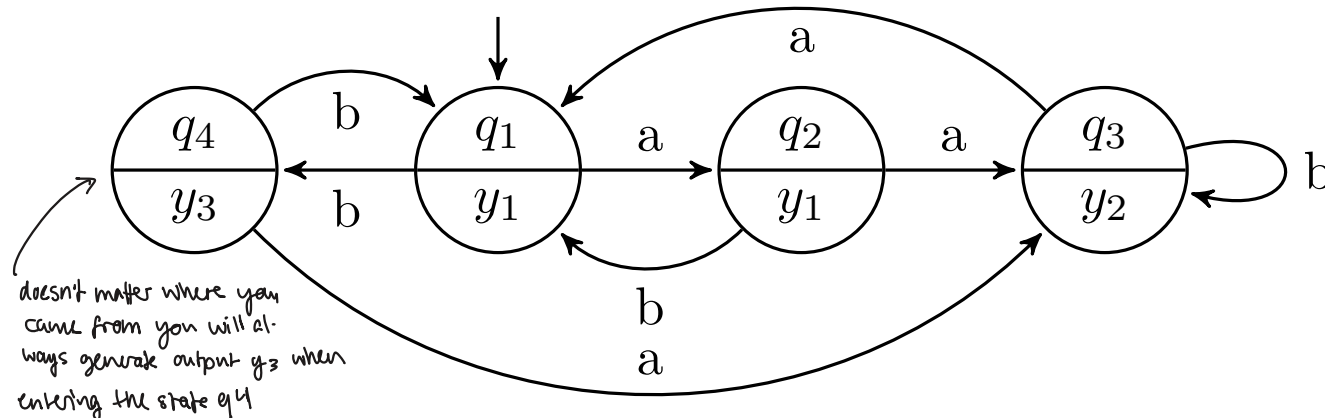
- Extend definition by
 - an additional alphabet of **output symbols** $Y = \{y_1, y_2, \dots\}$
 - an additional mapping g for generating the output
- **Mealy** machine – use $g: Q \times \Sigma \rightarrow Y$
 - output symbol depends on input symbol and current state



σ_i	q_1	q_2	q_3
a	q_2, y_1	q_3, y_2	q_1, y_1
b	q_2, y_3	q_1, y_1	q_3, y_2



- Extend definition by
 - an additional alphabet of **output symbols** $Y = \{y_1, y_2, \dots\}$
 - an additional mapping g for generating the output
- **Moore** machine – use $g: Q \rightarrow Y$
 - output symbol depends only on current state

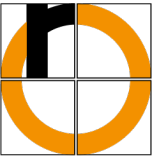


σ_i	q_1	q_2	q_3	q_4
a	q_2, y_1	q_3, y_2	q_1, y_1	q_3, y_2
b	q_4, y_3	q_1, y_1	q_3, y_2	q_1, y_1

- Extend definition by
 - an additional alphabet of **output symbols** $Y = \{y_1, y_2, \dots\}$
 - an additional mapping g for generating the output
- **Mealy** machine – use $g: Q \times \Sigma \rightarrow Y$
 - output symbol depends on input symbol and current state
- **Moore** machine – use $g: Q \rightarrow Y$
 - output symbol depends only on current state
- For each input sequence an output sequence is generated
- The automaton translates input to output $\rightarrow$ transducer (*Transduktor*, ***übersetzender Automat***)
- If Q, Σ, Y are finite sets: **finite transducers** (*endlicher Übersetzer*) translates input & output
- Both definitions are equivalent – they can perform the same tasks (eg Java input $\rightarrow$ machine output)

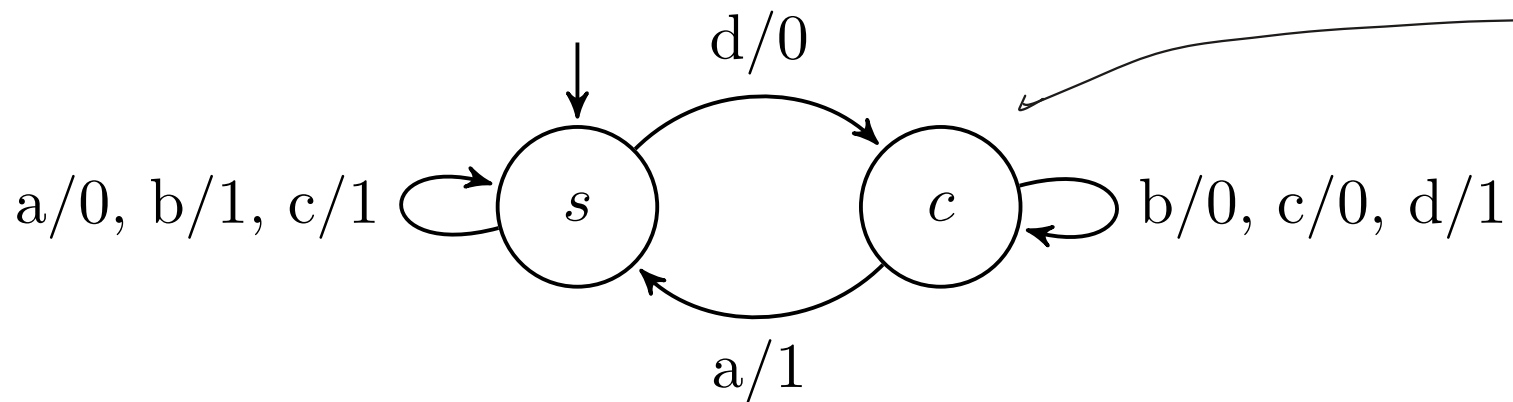
- Some problems require more than one input sequence
- By definition, an automaton can only process one input sequence
- Solution:
 - Redefine the combinations of symbols that appear simultaneously as new symbols of an extended alphabet
 - Construct an automaton that again processes only a single input sequence

Multiple Inputs – Example



Add two binary numbers

- original input alphabet: $\Sigma = \{0, 1\}$
- define new input alphabet $\Sigma' = \{a, b, c, d\}$ with
a: $0 + 0 = 0$, b: $0 + 1 = 1$, c: $1 + 0 = 1$, d: $1 + 1 = 0$ with carry 1
- output alphabet: $Y = \{0, 1\}$ (result of adding two digits)
- states: carry (c) or no carry (s)

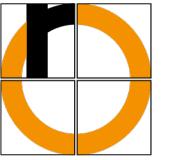


0 1 0 1 0
+ 0 0 1 1 1

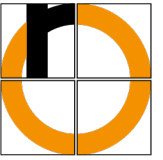
a c b d b

numbers I want to add
translate to new alphabet
read from right to left since addition happens in that order

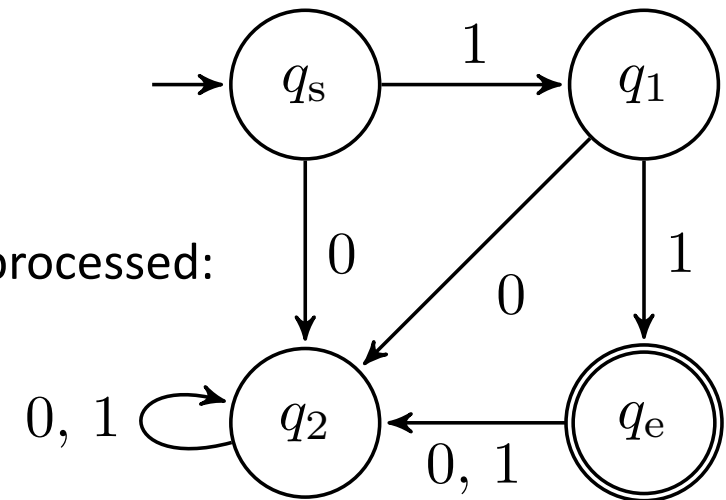
Read b d b c a
output 1 0 0 0 1

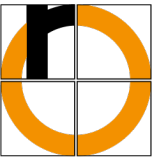


Acceptors: Recognized Language of Finite Automata



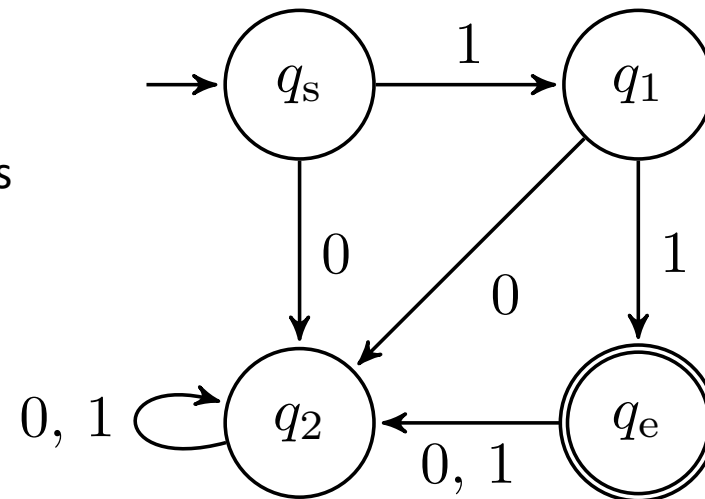
- Again, we consider automata without output and extend the definition by:
- the set of **accept states** E (or **end states**, *akzeptierende Zustände bzw. Endzustände*)
 - $E \subseteq Q$, marked by two concentric circles in the state diagram
- This is an **acceptor** (*Akzeptor, erkennender Automat*)
 - Start processing of input string (left to right) at start state
 - Follow transitions for each symbol
 - If the automaton is in an end state after all symbols have been processed:
The input string ("word") is said to be accepted.



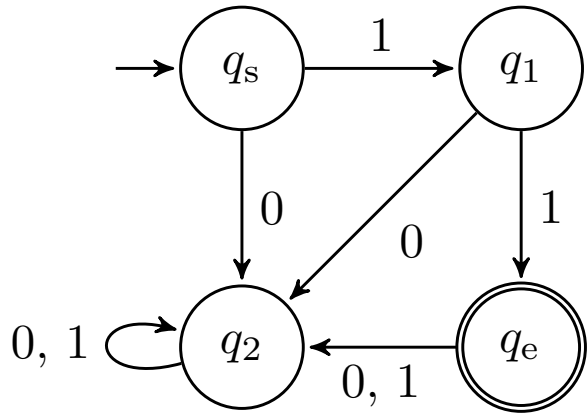
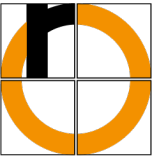


- Set of all input words w accepted by an automaton A (i.e., it stops in an end state):
recognized language $L(A)$ (*akzeptierte Sprache*)
$$L(A) := \{w \in \Sigma^* \mid (q_s, w) \rightarrow \cdots \rightarrow q_e, q_s = \text{start state}, q_e \in E\}$$
- Cardinality (*Mächtigkeit*) of L : $|L|$ = number of words of the language
- Two automata are said to be **equivalent** if they recognize the same language

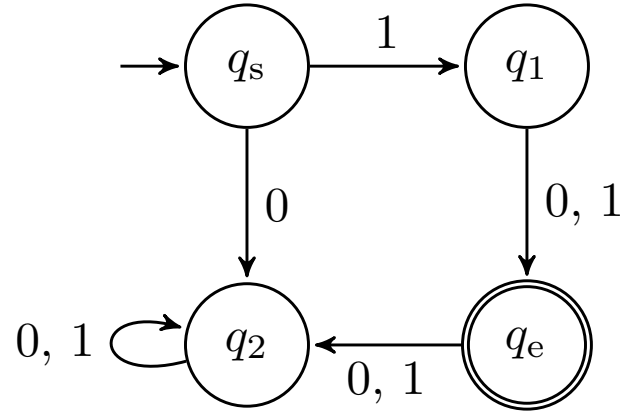
Recognized language contains
only a single word:
 $L = \{11\}, |L| = 1$



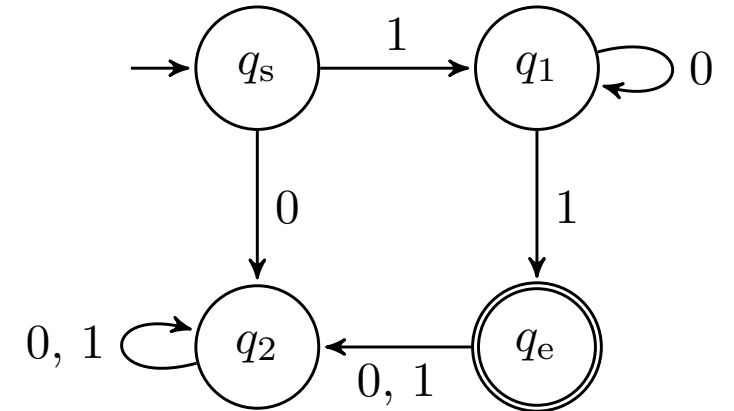
Recognized Language – Examples



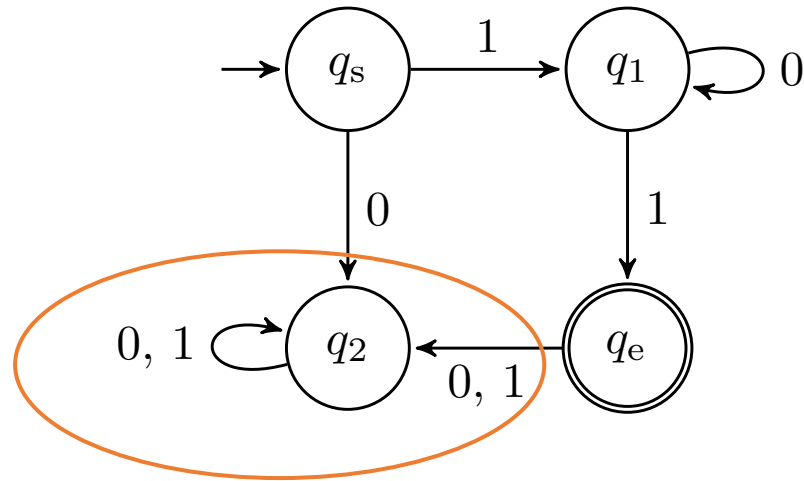
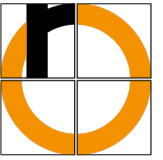
$$L = \{11\}, |L| = 1$$



$$L = \{11, 10\}, |L| = 2$$

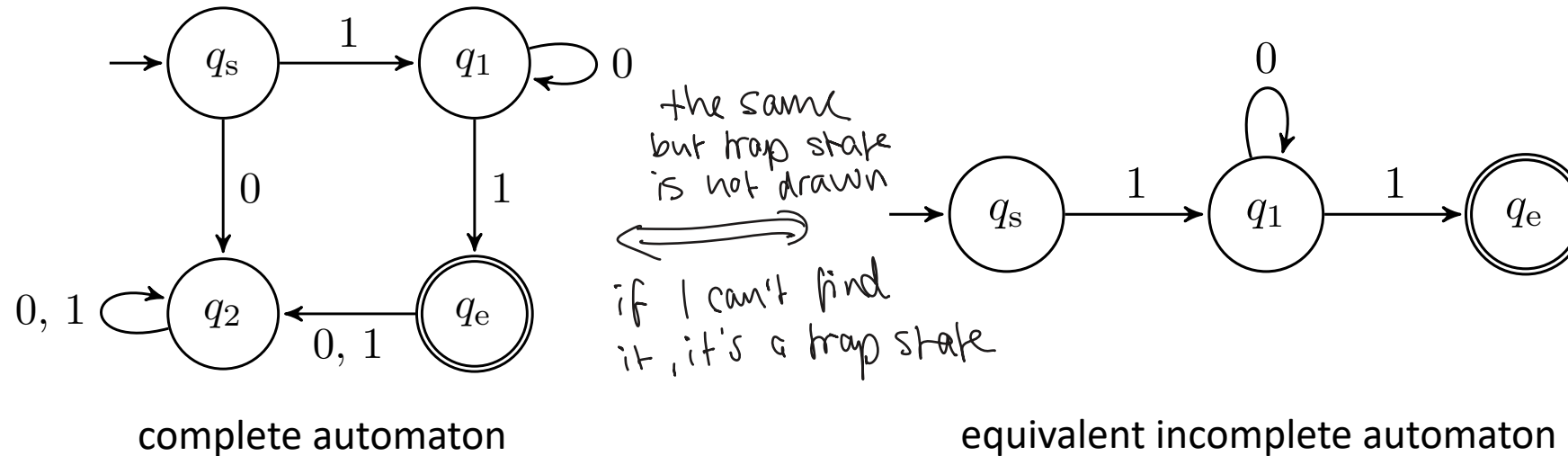


$$L = \{10^n 1 \mid n \in \mathbb{N}_0\}, |L| = \infty$$



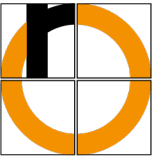
Trap state: Once reached, the automaton can never escape again – the processed word will not be part of the language, no matter which symbols follow. A trap state is **never** an end state!

These states are typically omitted for reasons of clarity → incomplete automaton.



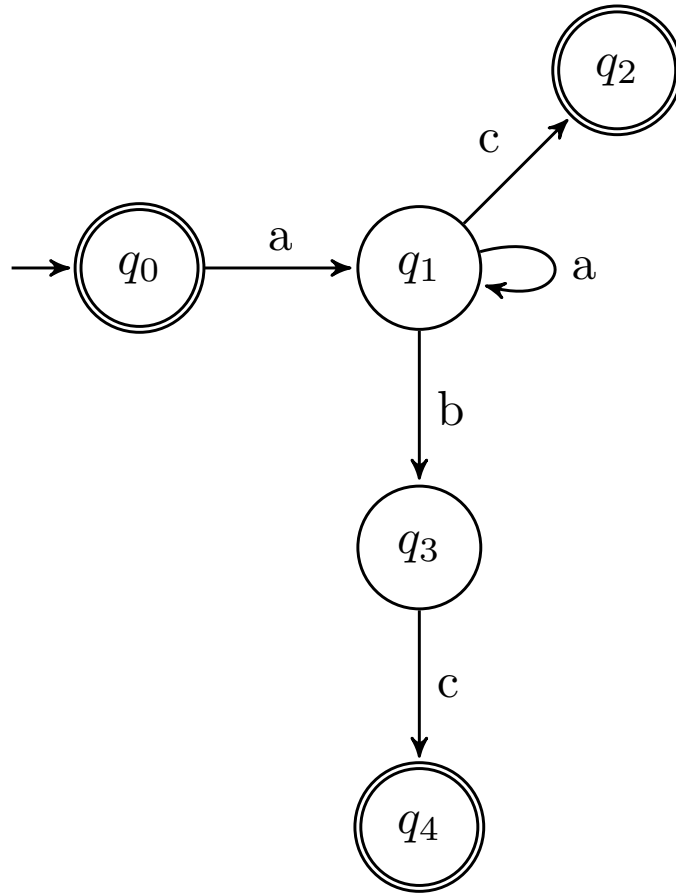
σ_i	q_s	q_1	q_e
0	—	q_1	—
1	q_1	q_e	—

- Trap states are usually omitted for clarity
- Convention: All transitions that are not shown end in a trap state
- In a compiler, these can be used to display error messages



What is the recognized language of the following automaton?

$$a^0 = \varepsilon$$



$$L = \{ \varepsilon, a^n b c, a^n c \} \quad n \in \mathbb{N} \neq 0, |L| = \infty$$

LÖSUNG:

$$L = \{ \varepsilon \} \cup \{ a^n c \mid n \in \mathbb{N}_0 \} \cup \{ a a^n b c \mid n \in \mathbb{N}_0 \} \\ \downarrow \{ a^n c \mid n \in \mathbb{N} \}$$

- The languages recognized by finite automata are identical to the regular languages
- **Regular language**: The subset of the word semigroup $(\Sigma^*, \circ)$ that can be generated by
 - string concatenation ($\circ$) = attach one string to another; symbol is usually omitted
 - set union ($\cup$)
 - Kleene closure ($*$)

- **Kleene closure** Σ^* of Σ (*Kleenesche Hülle*): $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

- contains all words that can be generated from the alphabet
- including the empty string ε

- **Kleene plus** Σ^+ of Σ (*positive Hülle*): $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

- ε is the neutral element of the semigroup

- Example: $\Sigma = \{a, b\}$

- $\Sigma^* = \{\varepsilon, a, b, aa, bb, ab, ba, aaa, \dots\}$

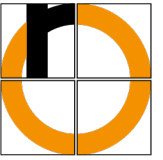
- $ab \circ abb = ababb$, $(aa \circ bb) \circ aba = aa \circ (bb \circ aba) = aabbaba$

$(\mathbb{N}, +)$

$(\mathbb{N}, +)$
semigroup

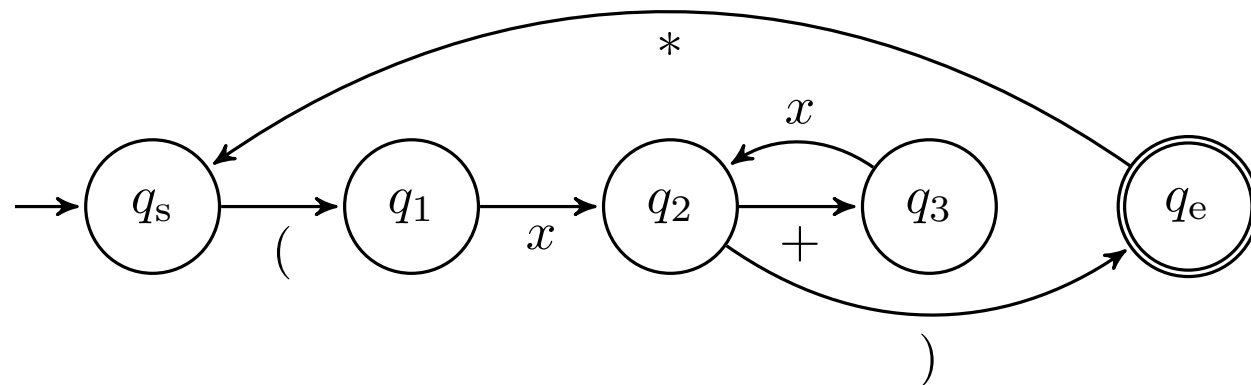
Set S $a, b, c \in S$
Operation $\circ$ $a \circ b \in S$
 $(a \circ b) \circ c = a \circ (b \circ c)$

- Word Problem:
Decide, whether an input string is part of the recognized language of an automaton.
- Application examples
 - Pattern matching: Finding a given string in a text
 - Lexical analysis in compilers
 - Decide, whether an input sequence (source code) adheres to the rules of a programming language
 - Split source code into elementary units (**tokens**), e.g., key words, identifiers, operators, ...

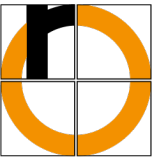


Analysis of simple arithmetic expressions with parentheses:

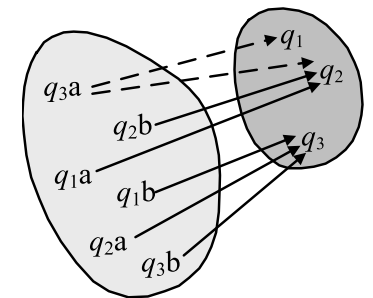
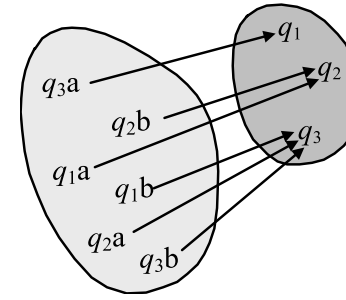
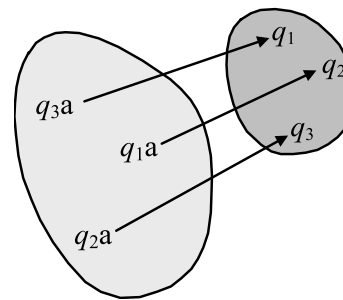
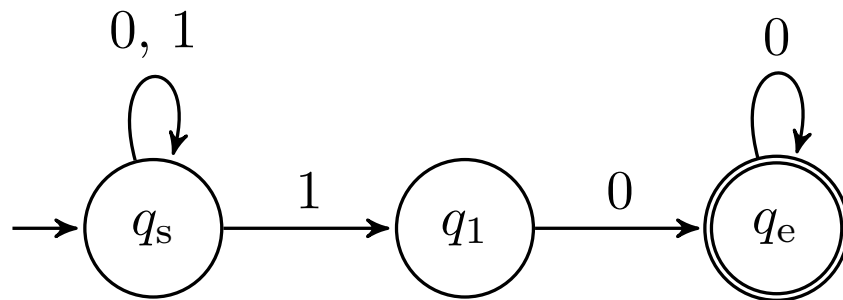
- Check, whether terms of the form $(x+x)^*(x+x+x)$ are built correctly
- Input alphabet: $\Sigma = \{ (,), +, *, x \}$



σ_i	q_s	q_1	q_2	q_3	q_e
(	q_1	—	—	—	—
)	—	—	q_e	—	—
+	—	—	q_3	—	—
*	—	—	—	—	q_s
x	—	q_2	—	q_2	—



- There is at least one state that has more than one transition with the same label
- Change the definition of the transition mapping from “function” to “relation”

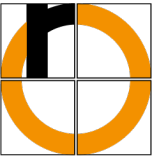


chooses the path that
leads to an end state after
processing the whole input string

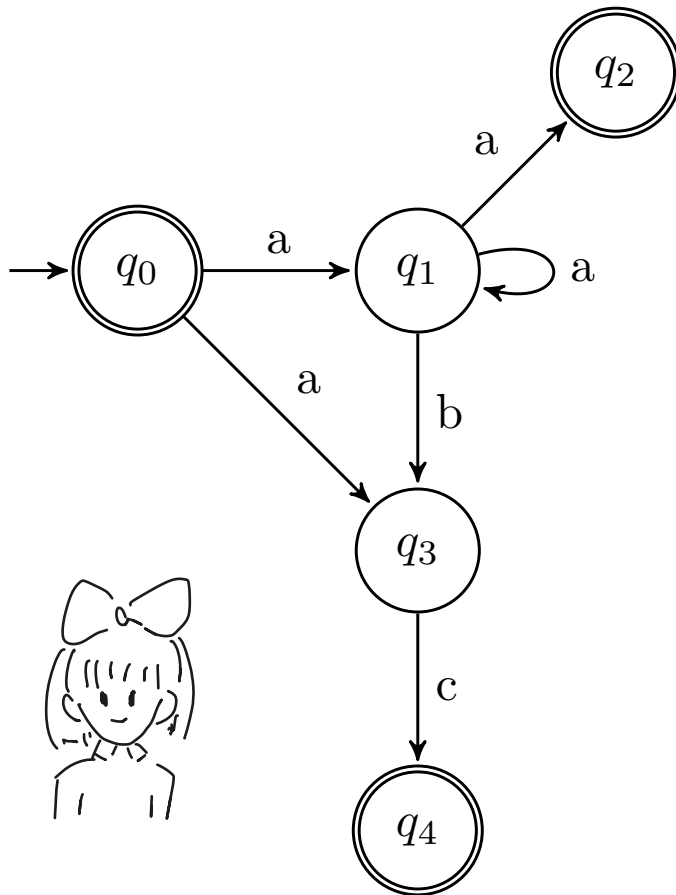
deterministic

nondeterministic

- The automaton always chooses the correct successor state, without looking ahead (i.e., such that an end state will be reached, if it can be reached at all)
- Surprising: For any NFA we can construct an equivalent DFA
 - a deterministic version accepting the same language
 - nondeterminism does not result in more computational power (but may be faster)



What is the recognized language of the following automaton?



$$L = \{\varepsilon, a^2, ac, aa^nbc\}$$

$$L = \{\varepsilon\} \cup \{a^2\} \cup \{ac\} \cup \{aa^nbc \mid n \in \mathbb{N}_0\}$$

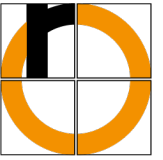
LÖSUNG:

$$\begin{aligned} &\{\varepsilon\} \cup \{aa^na \mid n \in \mathbb{N}_0\} \\ &\cup \{aa^nbc \mid n \in \mathbb{N}_0\} \\ &\cup \{ac\} \end{aligned}$$

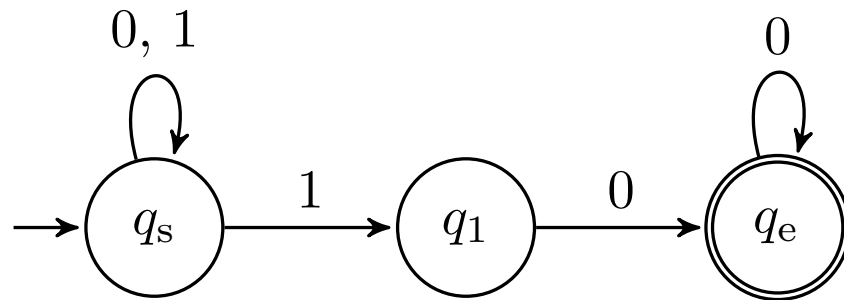
- For any nondeterministic finite acceptor (NFA), an equivalent deterministic one (DFA) can be constructed that recognizes the same language
- In 1976, M. Rabin and D. Scott received the [Turing Award](#) for the proof
- Basic idea:
 - given: NFA with n states
 - Determine all transitions of subsets of the state set Q
 - Set of all subsets: **powerset** $\wp(Q)$ (*Potenzmenge*); contains 2^n elements
 - the resulting DFA can thus have more states and transitions than the original NFA
 - in most cases, however, the number of states grows only slightly or even remains the same

set of all subsets eg.: $A = \{a, b\}$
 $\{ \{\emptyset\}, \{a\}, \{b\}, \{a, b\} \}$
 $\hookrightarrow$ warum nicht $\{b, a\}$

Transformation NFA $\rightarrow$ DFA – Example



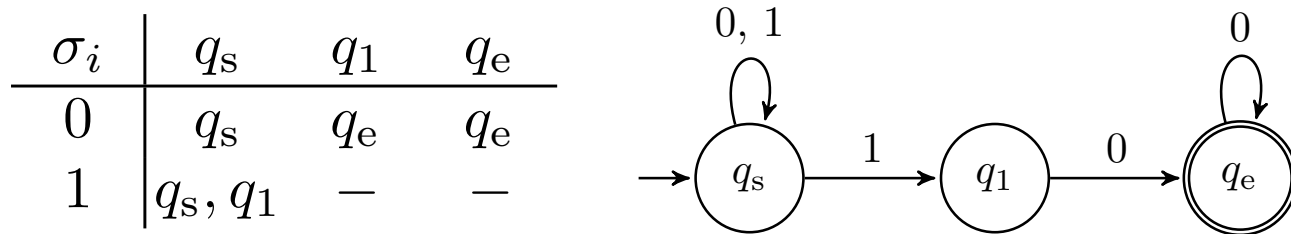
- given: NFA with $\Sigma = \{0, 1\}$, $Q = \{q_s, q_1, q_e\}$
- recognized language: $L = \{x10^n \mid x \in \Sigma^*, n \in \mathbb{N}\}$



σ_i	q_s	q_1	q_e
0	q_s	q_e	q_e
1	(q_s, q_1)	—	—

more than one target state
 $\rightarrow$ non-deterministic automata

Transformation NFA $\rightarrow$ DFA – Example



(1) Start with all sets of successor states that can be reached directly from start state set $\{q_s\}$

σ_i	$\{q_s\}$
0	$\{q_s\}$
1	$\{q_s, q_1\}$

(2) Add all newly listed subsets as new columns

σ_i	$\{q_s\}$	$\{q_s, q_1\}$
0	$\{q_s\}$	$\{q_s, q_e\}$
1	$\{q_s, q_1\}$	$\{q_s, q_1\}$

where can I go from q_s with either a 1 or a 0
→ answer q_s

where can I go from q_1 with either a 1 or a 0
→ answer q_e

(3) Iterate until no new columns must be added any more

σ_i	$\{q_s\}$	$\{q_s, q_1\}$	$\{q_s, q_e\}$
0	$\{q_s\}$	$\{q_s, q_e\}$	$\{q_s, q_e\}$
1	$\{q_s, q_1\}$	$\{q_s, q_1\}$	$\{q_s, q_1\}$

All subsets containing an end state of the NFA are end states of the DFA

Transformation NFA $\rightarrow$ DFA – Example

(4) Rename states (if desired)

σ_i	$\{q_s\}$	$\{q_s, q_1\}$	$\{q_s, q_e\}$
0	$\{q_s\}$	$\{q_s, q_e\}$	$\{q_s, q_e\}$
1	$\{q_s, q_1\}$	$\{q_s, q_1\}$	$\{q_s, q_1\}$

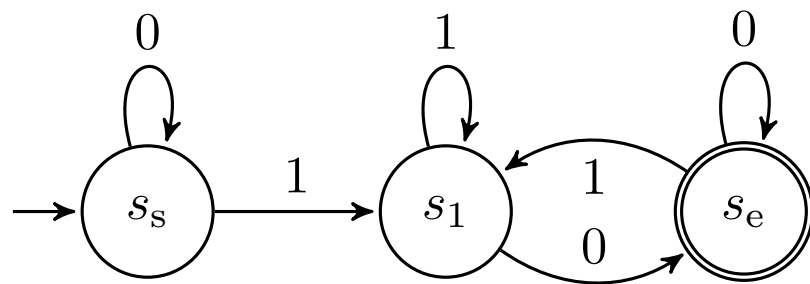
 $\longrightarrow$

σ_i	s_s	s_1	s_e
0	s_s	s_e	s_e
1	s_1	s_1	s_1

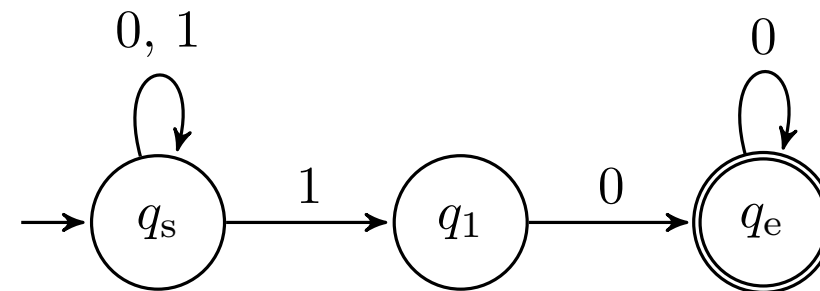
Out of a total of 8 possible states of the power set only 3 are actually used. Power set:

$$\wp(Q) = \{\{\}, \{q_s\}, \{q_1\}, \{q_e\}, \{q_s, q_1\}, \{q_s, q_e\}, \{q_1, q_e\}, \{q_s, q_1, q_e\}\}$$

Resulting state diagram of DFA:



State diagram of original NFA:

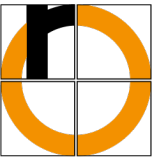


$$L = \{x10^n \mid x \in \Sigma^*, n \in \mathbb{N}\}$$

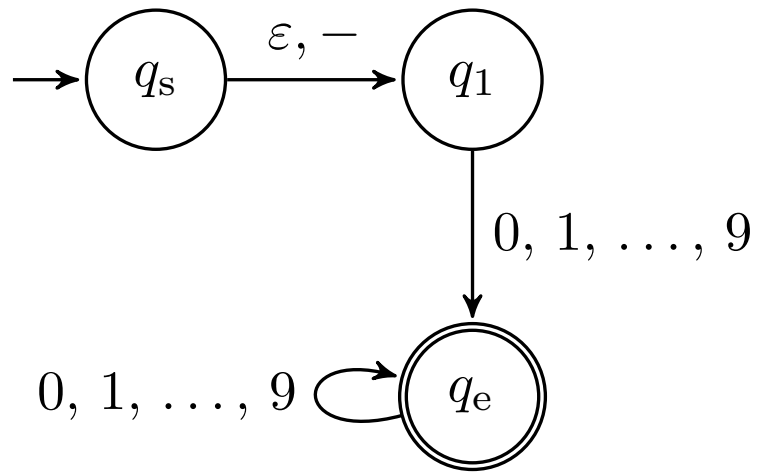
- Given: incomplete NFA
- Write all the input symbols in the first column of a table.
- Write the subset that contains all the initial states of the NFA as a header in the second column of the table. This is the initial state of the DFA.
- In the rows of the second column, enter the subsets that are reached from the initial state when the appropriate input symbol is entered.
- Add the new subsets in this column as new column headers to the table.
- In the rows of these columns, enter the subsets that are reached from the subsets in the column headers with the respective input symbols.
- Iterate until no new subsets/columns occur.
- The subsets found correspond to the states of the DFA.
- The transitions of the DFA result directly from the table entries.
- All subsets containing an end state of the NFA are end states of the DFA.
- For clarity, the state subsets can be renamed.

In any case, this algorithm stops after a finite number of steps, namely after a maximum of 2^n , since this corresponds to the cardinality of the power set.
 $\rightarrow$ Time complexity $O(2^n)$

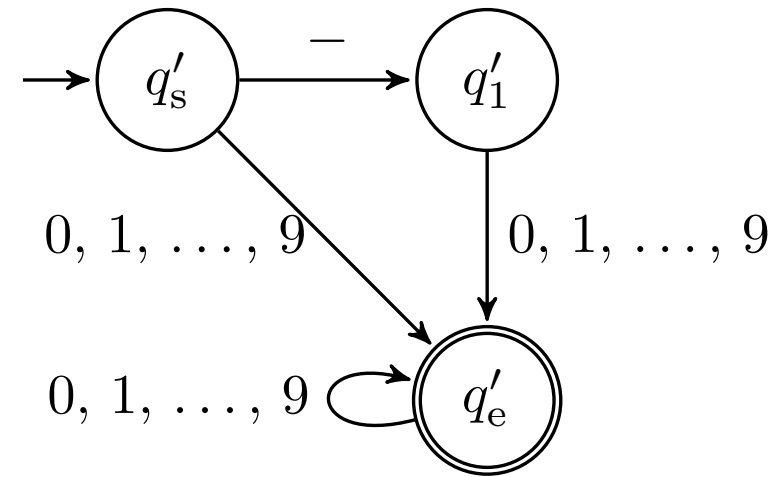
- DFA is **deterministic**: transition from the current state to the subsequent state is unambiguous
- DFA and NFA are also **causal**: a transition is determined by a specific input symbol
- Causality can be restricted: Allow **spontaneous state transitions** (not caused by input symbols)
- These are called **ε -moves** (or: λ -moves)
 - where ε is the empty string (of length 0)
 - ε is not an element of the input alphabet, but rather part of the transition function itself
- **For any NFA with ε -moves, an equivalent DFA can be constructed containing no such transitions**
 - The construction is the same as the Rabin-Scott algorithm for transforming NFA to DFA
 - with special treatment for ε -moves (use ε -closures, see textbooks for details)



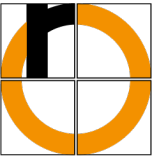
Automaton recognizing integers, with an optional minus-sign



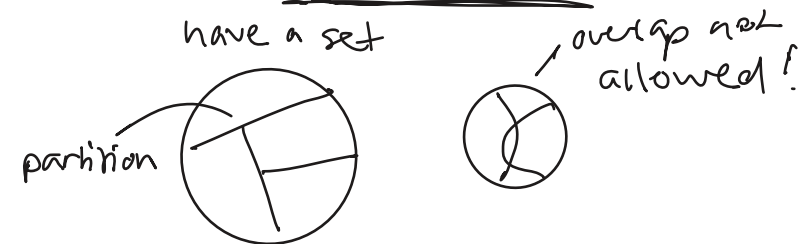
with ε -moves



without ε -moves

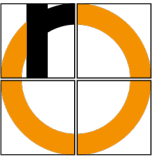


- The minimal DFA
 - is the one with the smallest number of states of all equivalent DFAs
 - is unique
 - can be constructed starting from any DFA (for NFA: convert to equivalent DFA first)
- Construction: Build **equivalency classes** (*Äquivalenzklassen*) by partitioning
 - Start with all states in a single partition
 - Separate end states from other states in a single partition
 - Split partitions until all states in a partition are equivalent



Minimal DFA – Example

works only for deterministic automats



Original transition table

σ_i	q_s	q_1	q_2	q_3	q_e
0	q_1	q_e	q_3	q_e	q_e
1	q_2	q_2	q_2	q_s	q_e



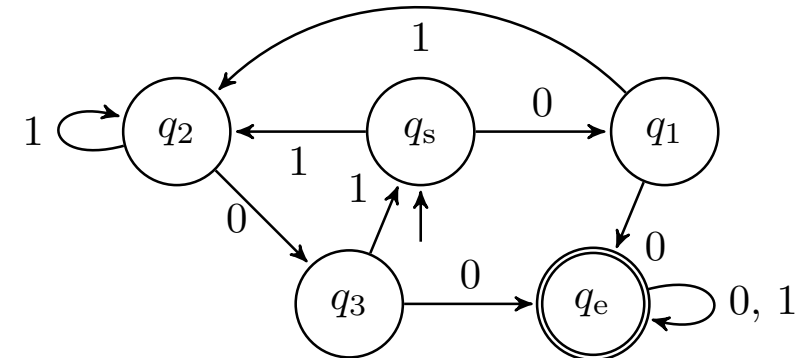
(1) Separate end states from other states

σ_i	P_1				P_2
	q_s	q_1	q_2	q_3	q_e
0	q_1, P_1	q_e, P_2	q_3, P_1	q_e, P_2	q_e, P_2
1	q_2, P_1	q_2, P_1	q_2, P_1	q_s, P_1	q_e, P_2

sign that I may be able to merge it

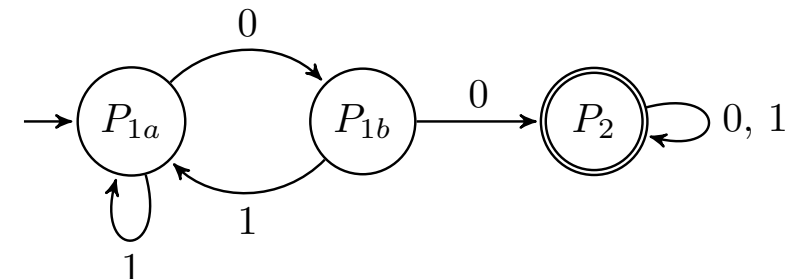
(2) Split each partition until all states are equivalent

σ_i	P_{1a}		P_{1b}		P_2
	q_s	q_2	q_1	q_3	q_e
0	q_1, P_{1b}	q_3, P_{1b}	q_e, P_2	q_e, P_2	q_e, P_2
1	q_2, P_{1a}	q_2, P_{1a}	q_2, P_{1a}	q_s, P_{1a}	q_e, P_2



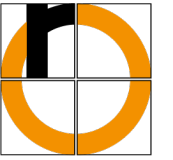
Result:

- all states of an equivalence class can be combined
- we get a DFA with 3 states:



- Automata with output: Both types are equivalent
 - Mealy: output at transition
 - Moore: output in state
- Accepting finite automata
 - process an input string (a “word”) from left to right
 - the word is accepted if the automata is in an end state after all symbols have been processed
 - the set of all accepted words is the recognized language
 - the recognized languages of finite automata are identical to the regular languages
 - incomplete automaton: trap states are omitted
- Deterministic (DFA) and nondeterministic (NFA) automata are equivalent
 - i.e., they have the same computational power (= they recognize regular languages)
 - an NFA may be faster though
 - ε -moves in NFAs allow for spontaneous transitions without cause
 - we can construct an equivalent DFA from any given NFA
 - we can construct an equivalent minimal DFA from any given DFA

- H. Ernst, J. Schmidt und G. Beneken: *Grundkurs Informatik*. Springer Vieweg, 8. Aufl., 2023.
- Schöning, U.: *Theoretische Informatik – kurz gefasst*. Spektrum Akad. Verlag (2008)
- Sander P., Stucky W., Herschel, R.: *Automaten, Sprachen, Berechenbarkeit*, B.G. Teubner, 1992

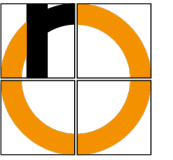


Theoretical Computer Science

Pushdown Automata & Turing Machines

Technische Hochschule Rosenheim
Sommer 2026
Prof. Dr. Jochen Schmidt

- Pushdown automata
- Turing Machines
- Linear bounded automata



Pushdown Automata

- A DFA/NFA can only recognize regular languages
 - formed by string concatenation, set union ($\cup$) and Kleene closure ($*$)
 - A DFA/NFA does not have memory
 - DFA/NFA cannot test strings with arbitrarily deep nested brackets for correctness
 - like parentheses in arithmetical expressions like $x = (((a + b) * c + (c + d) * (a + c))) * d;$
 - or nested block structures in C or Java with braces $\{...\{...\}...\{...\}...\}$
- Extension of NFA by adding memory in form of a **stack** (*Kellerspeicher*)
- a stack has a bottom and is unlimited in the other direction
 - only the element at the top can be accessed directly: push/pop operations
 - **Pushdown Automaton** (PDA, *Kellerautomat*)

Extension of NFA by a **stack** and

- a finite **stack alphabet** Γ Greek Capital Gamma
- an **initial stack symbol** $\#$ marking the bottom of the stack
- extension of the transition mapping
 - a transition depends on the current input symbol as well as the symbol at the top of the stack
 - in each transition
 - the top stack symbol is removed (**pop**)
 - none, one or multiple symbols can be written to the top of the stack (**push**)
- This general definition is nondeterministic – as with finite automata, we can restrict it to deterministic transitions and get a Deterministic PDA or DPDA.

- Words can be accepted using
 - **end states** (same as for finite automata), independent of the content of the stack
 - **empty stack** – no end states, a word is accepted if the stack is completely empty after the input sequence has been processed (including the initial stack symbol #)
- These two options are equivalent for nondeterministic PDAs
 - They are different for DPDAs: Accepting words by end states is more powerful
- Nondeterministic PDAs recognize the so-called **context-free languages**
 - these are a superset of the regular languages

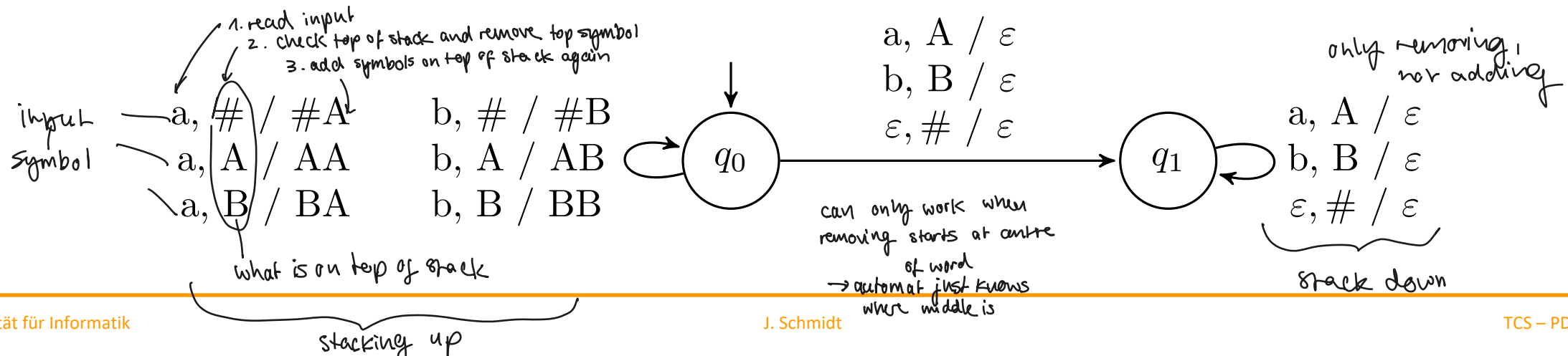
nondeterministic $\Leftrightarrow$ deterministic
 $\rightarrow$ more powerful

- Correct nesting: { { } { } }
- Incorrect nesting: { } } { } { – just counting opening/closing braces will not work!
- Input alphabet $\Sigma = \{\{, \}\}$, stack alphabet $\Gamma = \{\#, \{ \}$
- Accept words by empty stack
- How the PDA works:
 1. Input symbol { : push it on top of stack.
 2. Input symbol } :
 - a) If the stack is in initial state (# on top): Error! The nesting is incorrect.
 - b) If the stack is not in initial state: Pop top { from stack; each } input removes one {.
 3. After all input symbols have been processed:
 - a) If the stack is in initial state: The block structure is correct. Pop #, the stack is now empty, the word accepted.
 - b) If the stack is not in initial state: Error! The block structure is incorrect.

This PDA is deterministic and has only a single state – no point in drawing a state diagram

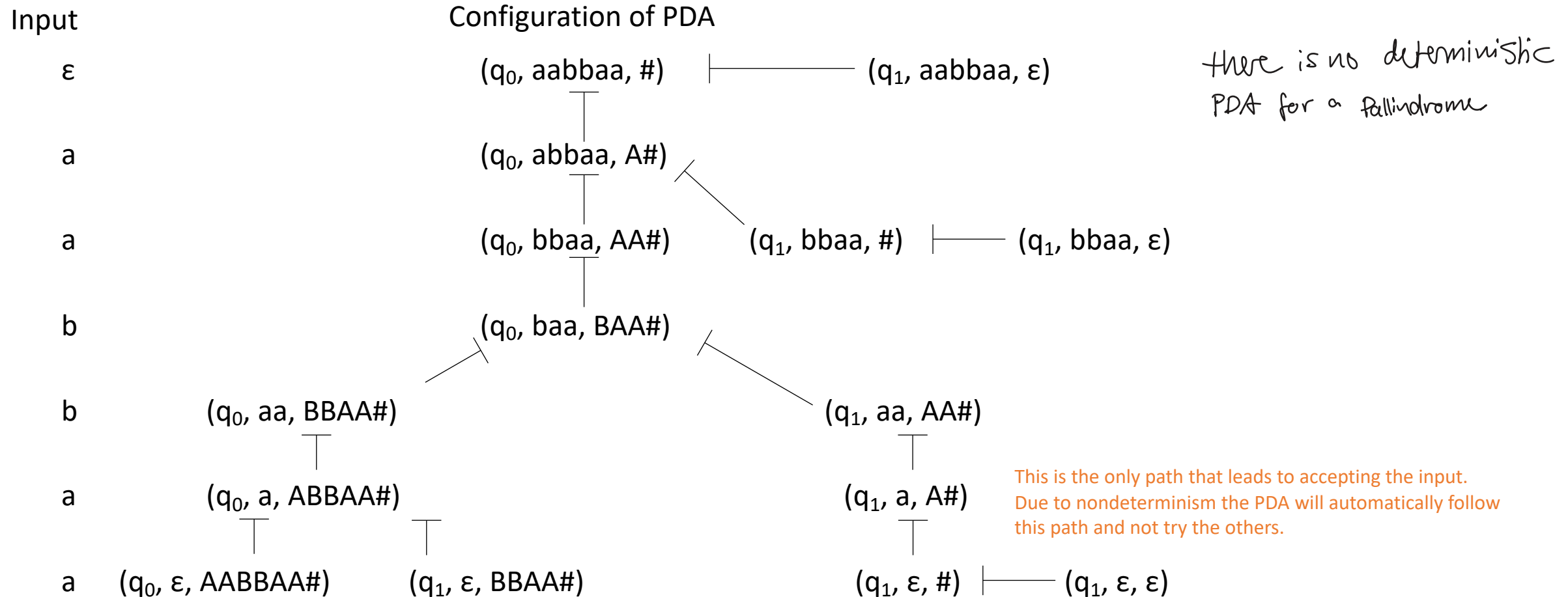
Example: Mirroring (Palindromes)

- Input alphabet $\Sigma = \{a, b\}$, stack alphabet $\Gamma = \{A, B, \#\}$
- Recognized language: palindromes (*Palindrome*): $L = \{x_1x_2 \dots x_nx_n \dots x_2x_1 \mid x_i \in \Sigma\}$
- Accept words by empty stack
- Transition notation: **a, # / #A** means:
 - Do the transition if a was read as an input symbol
 - and # is on top of the stack (and will be popped).
 - Push #A on top of the stack.
 - (so, this example will actually leave # on top and in addition push A)



Example: Mirroring – Some Configurations

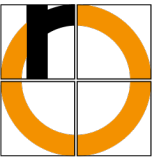
Some selected possible (not complete!) configurations when processing the input string aabbbaa



- The PDA for $L = \{x_1x_2 \dots x_nx_n \dots x_2x_1 \mid x_i \in \Sigma\}$ is nondeterministic.
- There exists no deterministic PDA (DPDA) that recognizes this language.
- The nondeterministic behavior is necessary to “guess” the middle of the word
- Only by marking the middle of the word we can construct a DPDA, e.g.:

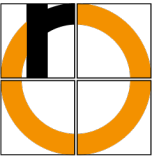
$$L = \{x_1x_2 \dots x_n8x_n \dots x_2x_1 \mid x_i \in \{a, b\}\}, \Sigma = \{a, b, 8\}$$

↑
middle of word



- Unlike DFA/NFA, PDA and DPDA are **not** equivalent: PDAs are more powerful than DPDAs
- DPDAs recognize only a proper subset of context-free languages:
the **deterministic context-free languages**
 - these are equivalent to the LR(k) languages ($k > 0$) and play an important role in compiler construction for syntax analysis
- What happens if we add more stacks?
 - A PDA with 2 stacks is computationally more powerful than a PDA with one stack
 - Adding yet more stacks may be more convenient, but does not increase computational power any further.
 - With 2 stacks a PDA is equivalent to a Turing Machine, the most powerful concept we know of.



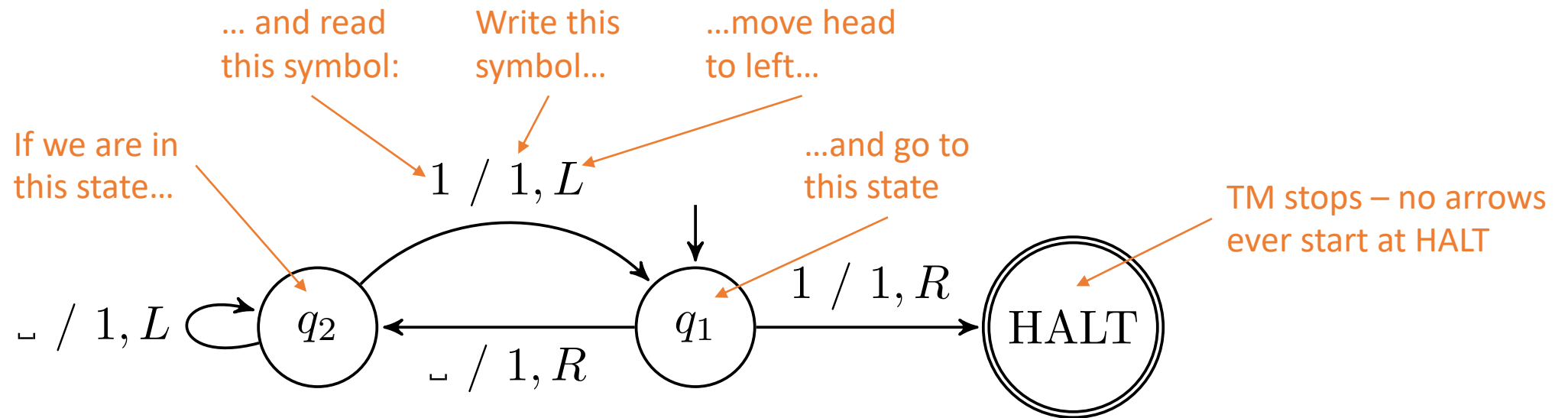
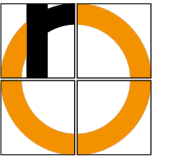


Turing Machines

- Finite automata and pushdown automata have obvious restrictions
 - DFA/NFA: no memory at all, only states
 - PDA: memory, where access is restricted by stack-principle (push/pop)
 - can recognize languages like $L = \{a^n b^n\}$, but not $L = \{a^n b^n c^n\}$
- Turing Machine: Use **memory tape** (*Band*) where we can move left & right to read/write
 - we will now read our input from this tape
- Developed by Alan Turing (1912 – 1954) in the 1930s
 - the ACM Turing Award is named after him (the "Nobel Prize" of computer science)
- Anything a computer can do, a Turing Machine can do – and vice versa
 - it provides a very simple model of a universal computer and is therefore often used in theoretical studies
 - all other known concepts for formulating algorithms or describing abstract computer models can be shown to be equivalent to Turing Machines

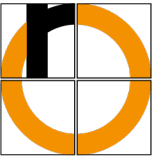
A (deterministic) Turing machine (TM) consists of

- an **infinite memory tape** for input and output (*Schreib-/Lese-Band*), divided into cells,
- a **read/write head** that can move along the tape by single steps to left (*L*) and right (*R*),
- a finite **input alphabet** Σ ,
- a finite **tape alphabet** Γ
 - Γ includes all input symbols and possibly additional ones, in particular the blank (space $\sqcup$) with which the band is filled at the beginning
- a finite set of **states** Q with one initial state and at least one end state (HALT state).
- a state **transition function** $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

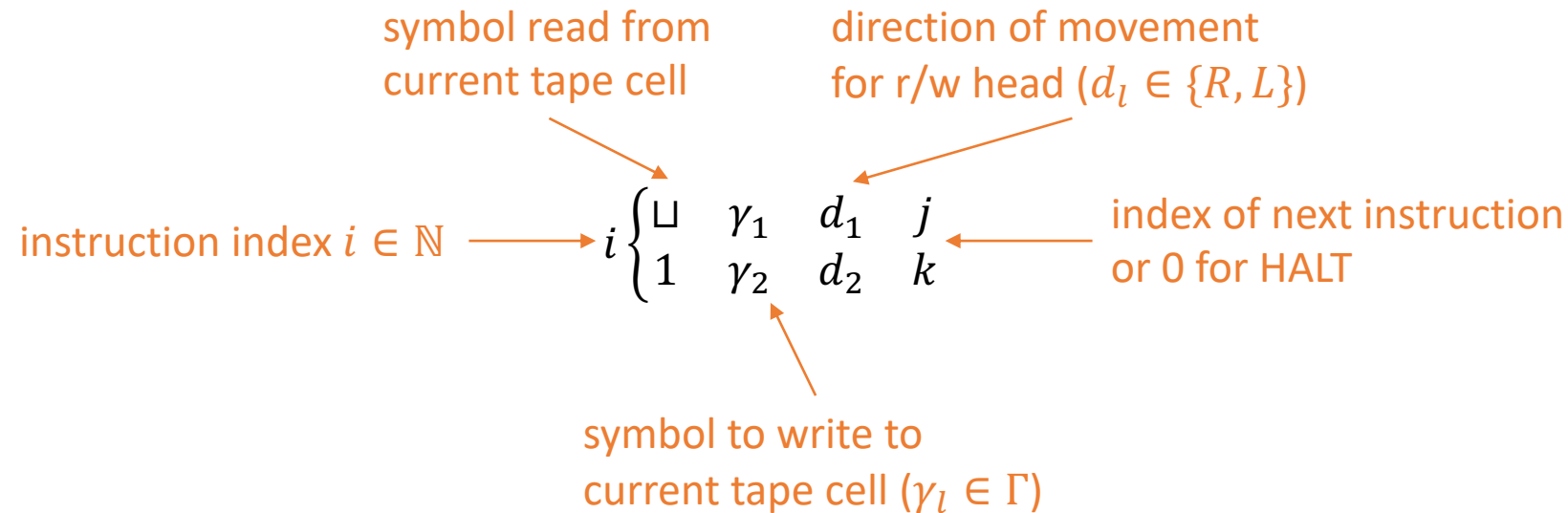


Additionally required:

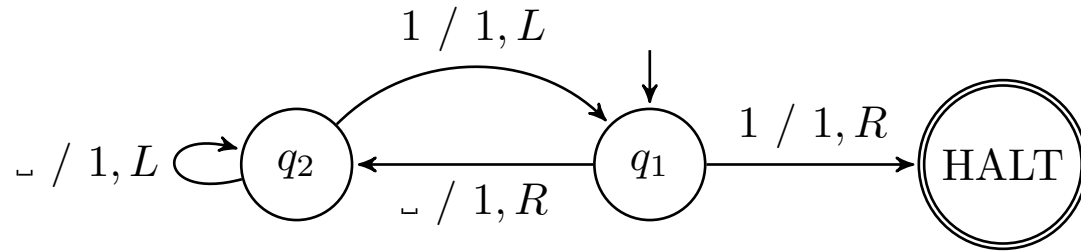
- Initialization of tape (typically: input string, remaining cells initialized with blank)
- Initial position of head



- Transition function of a TM is typically described by a finite number of **instructions**
- Example of structure (for tape alphabet limited to $\{\sqcup, 1\}$):

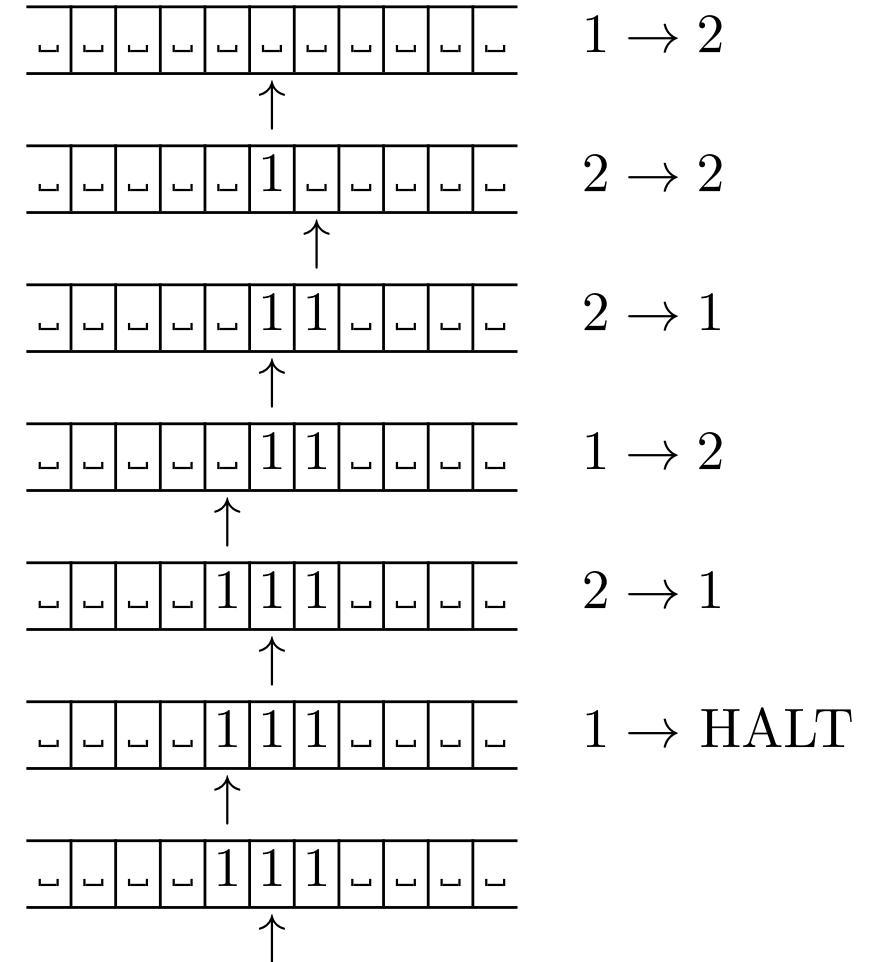


TM that writes three ones on tape initialized with blanks:



TM written using instructions:

$$1 \left\{ \begin{array}{cc} _ 1 R & 2 \\ 1 1 R & \text{HALT} \end{array} \right. \quad 2 \left\{ \begin{array}{cc} _ 1 L & 2 \\ 1 1 L & 1 \end{array} \right.$$



- Configuration = current symbols on tape + current state + current head position
 - Initial/Start configuration: configuration before processing is started
 - End/Halt configuration: configuration when TM has stopped
- Recognized language: Set of all input words where the TM stops in HALT state
 - TMs recognize the recursively enumerable languages
- Note:
 - A DFA/NFA/DPDA/PDA will always stop processing after it reaches the end of the input
 - A TM, however, may go into an infinite loop and never stop

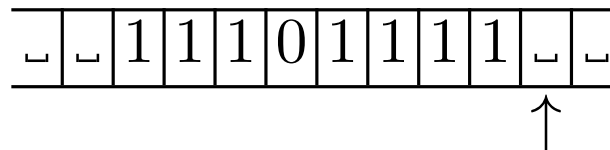
Example: “Unary” Addition

TM that can add two integers in “unary” notation:

- integer x = represented by x ones, e.g., $3 = 111$
- summands separated by 0, e.g., $3 + 2 = 111011$
- tape alphabet $\Gamma = \{\sqcup, 0, 1\}$
- initial head position: anywhere to the right of the input string

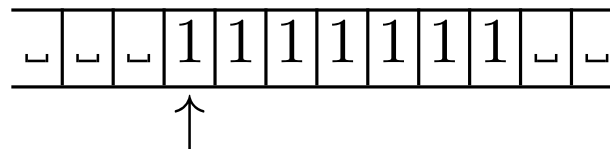
$$1 \begin{cases} \sqcup \sqcup L 1 \\ 0 1 L 2 \\ 1 1 L 1 \end{cases} \quad 2 \begin{cases} \sqcup \sqcup R 3 \\ 0 0 L \text{ HALT} \\ 1 1 L 2 \end{cases} \quad 3 \begin{cases} \sqcup \sqcup L \text{ HALT} \\ 0 0 L \text{ HALT} \\ 1 \sqcup R \text{ HALT} \end{cases}$$

Initial configuration

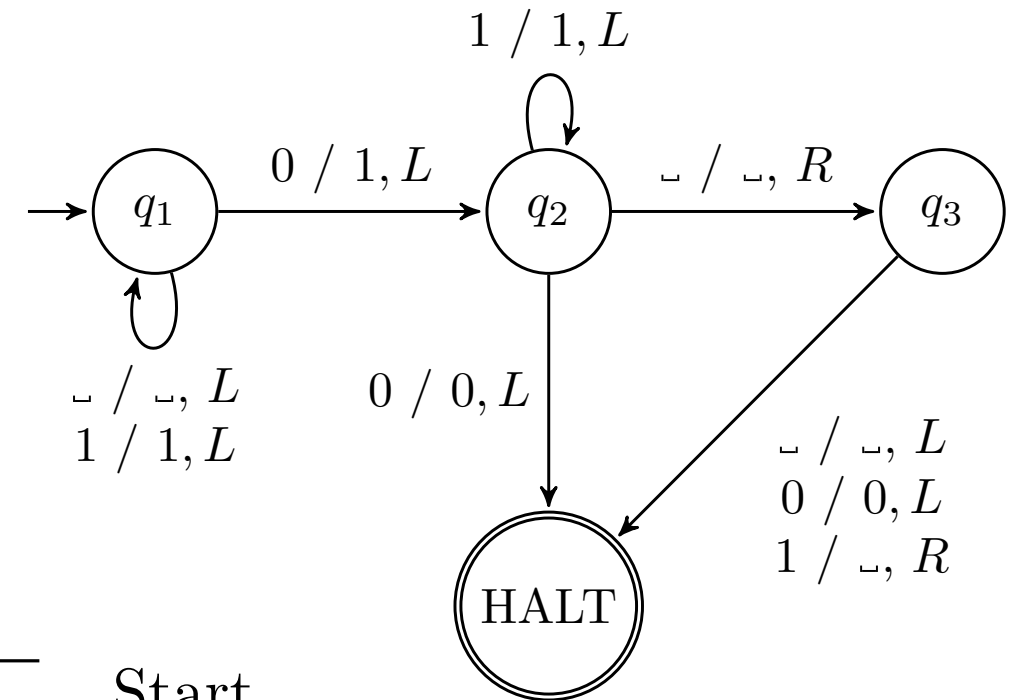


Start

Halt configuration



HALT



- Can be defined similar to finite/pushdown automata
- The NTM will always choose a transition that leads to Halt, if it exists
- Depending on your point of view a NTM
 - can guess the correct transition without looking ahead
 - or process all possibilities in parallel and select the correct one at the end
(this is not equivalent to parallel processing in computing – we have exponential growth)
- It can be proven that any **nondeterministic** TM can be converted to an **equivalent deterministic** TM (DTM) – a DTM is just as powerful as a NTM (but maybe slower)

We can actually restrict our TM definition without losing power. It has been proven:

- An alphabet with only **two symbols** (like 0, 1) is **sufficient** to do anything a TM can do
 - you may just need more states and more steps
- **Two states** are **sufficient** to do anything a TM can do (initial and halt state)
 - but you may need a larger alphabet
- A tape that is infinite in both directions is not required – one-directional infinity suffices

What if we restrict the tape of TM in both directions?

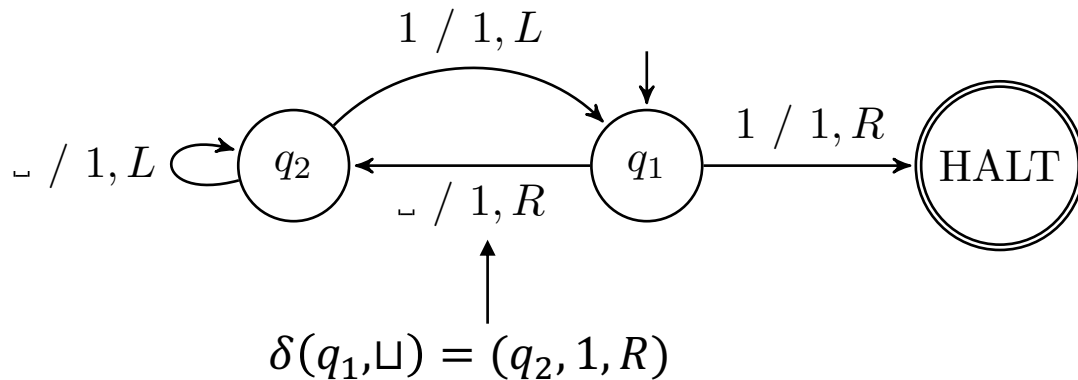
- Restrict tape length **to length of input string**
 - Finite memory, amount depending on input
 - We can still move left & right
- We get a **Linear Bounded Automaton** (LBA, *Linear beschränkter Automat*)
 - An LBA is less powerful than a TM; but still more powerful than a PDA!
- Whether nondeterministic LBAs are equivalent to deterministic LBAs is an open problem
- Recognized language: Set of all input words where the LBA stops in HALT state
 - LBAs recognize the **context-sensitive languages**
 - these are a proper
 - superset of the context-free languages
 - subset of the recursively enumerable languages

- **There is no known extension that makes the TM concept more powerful**
 - we have reason to believe that there is no such concept → Church-Turing thesis
 - extensions may just make processing more convenient/faster – but you cannot solve more problems
- **For example, you can**
 - add a neutral (N) position for head movement (i.e., the head does not move)
 - add multiple tapes with multiple r/w heads
 - let the r/w head move by more than a single cell
- **There is a multitude of very diverse concepts regarding models of computation:**
Until now, they have all been proven to be equivalent to Turing Machines
 - in particular: models with random access memory, as in real computers

- Universal Turing Machine (UTM) = TM that can simulate any other TM
 - A computer is basically a universal TM
 - Construction described by Alan Turing in 1936
- Therefore, any algorithm can be described as a TM and be executed by a universal TM
- A system that can simulate any TM is called **Turing-complete** (*Turing-vollständig*)
- Such a UTM surely is very large and complex? No! The smallest UTMs found have:
 - 4 states with 6 symbols and 22 instructions
 - 5 states with 5 symbols and 22 instructions
 - 15 states with 2 symbols and 29 instructions

- For simplification, consider the UTM having 3 tapes
 - **Coding tape**: Contains the encoding of the TM T to be simulated (its Gödel number)
 - **Operating tape**: Initially contains the input string for T , which is being processed on this tape
 - **State tape**: Used for storing the state the TM T would be in at each step
- **Gödel number of T** : Any injective mapping of TM T to natural numbers, where the inverse of the mapping can be computed
 - there are infinitely many ways of doing Gödel numbering
 - within a system the Gödel number for an instruction table is unique (otherwise: no inverse)
 - but not all natural numbers necessarily represent a TM (no one-to-one mapping)

Gödel Numbering – Simple Example



Encode each such transition as

$$b(q_1)1b(\sqcup)1b(q_2)1b(1)1b(R) = 01001001010$$

$$b(q_2)1b(1)1b(q_1)1b(1)1b(L) = 00101010100$$

- $b(\cdot)$: mapping of states/symbols/directions to unary representation
- 1: separator

Concatenate all encoded transitions $111E_111E_211 \dots 11E_4111$

This is the **Gödel number** of TM T

111010010010101100101010100111 ...

State/Symbol/Direction	Encoding $b(\cdot)$
q_1	0
q_2	00
HALT	000
1	0
$\sqcup$	00
R	0
L	00

- A TM can perform any calculations that a computer can do
 - all restrictions on TM also apply to real computers
- In principle, a TM has an infinite amount of memory available, a computer does not
 - but: in finite time, a TM can only process a finite amount of data
- TM allow statements about algorithms independent of real computers
 - these will always remain true, regardless of changes in the architecture of computers
- A deterministic TM is much slower than a real computer
 - but: Time differences are bounded by polynomial factors, so this is not relevant in principle (see also: Chapter on time complexity)

Deterministic Automaton	Nondeterministic Automaton	Are these equivalent?
DFA	NFA	yes
DPDA	PDA	no
DLBA	LBA	open problem
DTM	NTM	yes

Recognized languages:

$\text{regular} \subset \text{context-free} \subset \text{context-sensitive} \subset \text{recursively enumerable}$

$\text{regulär} \subset \text{kontextfrei} \subset \text{kontextsensitiv} \subset \text{rekursiv aufzählbar}$

DFA

PDA

LBA

DTM

NFA

NTM

So, we'll always use a Turing Machine, as it can do anything, right?

No: We'll try to use the simplest model available that can solve a problem

- the simpler the automaton model, the easier to handle
- computation time is typically larger for more general models (for example: see "word problem" in the next chapter)

- H. Ernst, J. Schmidt und G. Beneken: *Grundkurs Informatik*. Springer Vieweg, 8. Aufl., 2023.
- Schöning, U.: *Theoretische Informatik – kurz gefasst*. Spektrum Akad. Verlag (2008)
- Sander P., Stucky W., Herschel, R.: *Automaten, Sprachen, Berechenbarkeit*, B.G. Teubner, 1992
- D.W. Hoffmann. *Theoretische Informatik*. Hanser, 4. Aufl., 2018.